

딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사



목차

A table of Contents

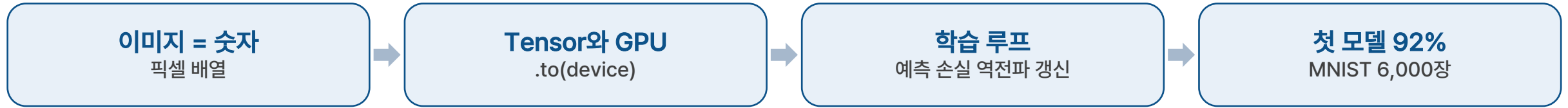
- 01** Full-batch 학습의 한계를 확인
- 02** Mini-batch와 Shuffle로 데이터를 공급
- 03** Dataset으로 데이터와 전처리를 관리
- 04** DataLoader로 배치를 만들어 공급
- 05** 학습과 검증 데이터를 분리 (Train, Val)
- 06** 완성한 파이프라인으로 모델을 다시 학습



PART 01

Full-batch 학습의 한계를 확인

1. 복습: 지난주에 우리가 한 것



- **지난주 학습한 내용**

- 이미지를 배열로 열어 전처리하고, Tensor로 바꿔 **GPU**로 보낸다
- 학습이 예측, 손실 계산, `backward()`, 파라미터 갱신의 반복임을 코드로 확인

- **오늘의 질문**

- 그런데 지난주엔 데이터 전체를 한 번에(full-batch) 넣었다, 그게 최선이었을까?

1. 지난주 방식의 문제 3가지

지난주 방식, full-batch 학습

```
# 지난주 6파트에서 한 방식 (전부 한 번에)
x = mnist.data[:6000].float().view(6000, 784) / 255 # 전처리를 직접
y = mnist.targets[:6000] # 6,000장 전체
x, y = x.to(device), y.to(device) # 전체를 GPU에
for epoch in range(301):
    loss = loss_fn(model(x), y) # 6,000장 전부 1회 갱신 (full-batch)
    ...
```

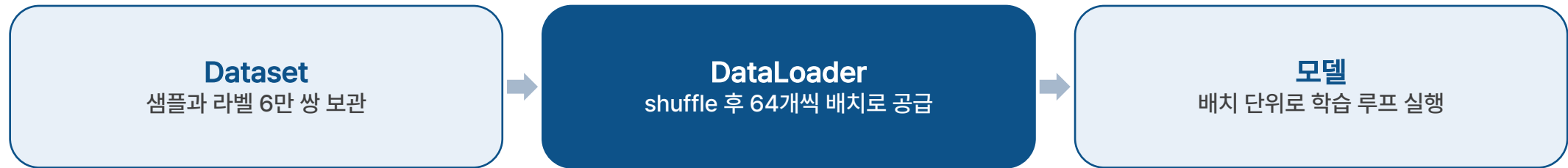
• Full-batch 방식의 문제 세 가지

- ImageNet(1,400만 장)이나 공정 검사 이미지 같은 실전 데이터는 메모리에 한 번에 올릴 수 없다.
- 전체를 다 계산해야 파라미터를 1회 갱신하므로 학습이 비효율적이다.
- `/255` 같은 전처리를 학습 코드 안에서 직접 반복해야 한다.

• 오늘

- 이를 해결하는 PyTorch의 표준 구성 요소 **Dataset**과 **DataLoader**를 학습

1. 오늘 만드는 것: 데이터 파이프라인



- 구성 요소는 딱 두 개

- **Dataset**은 샘플과 라벨을 보관하고 꺼내 주는 데이터 저장소다 (ex. 문제집)
- **DataLoader**는 데이터를 섞어 **batch** 단위로 공급하는 로더다 (ex. 컨베이어)
- 실무와 논문 코드가 전부 이 구조를 쓰며, 오늘 이 파이프라인으로 지난주 모델을 재학습해 **92%**를 넘겨 본다.

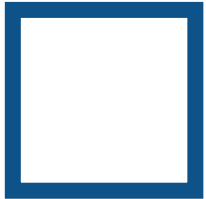
1. 환경 확인

확인 셀, 두 줄이 나오면 준비 끝

```
import torch, torchvision
from torch.utils.data import DataLoader      # 오늘의 핵심 구성 요소
device = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.__version__)      # → ex. 2.13.0+cu126
print(device)                 # → ex. cuda   (GPU가 없으면 cpu)
```

- 지난주와 같은 환경

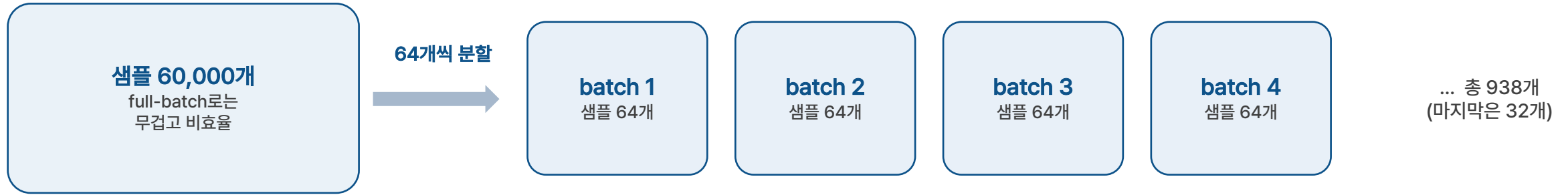
- 실습실 PC에서 `conda activate dl` 후 `jupyter notebook`을 실행한다 (또는 Colab)
- 지난주 노트북과 같은 폴더에서 시작하면 MNIST 다시 받을 필요 없음.
- `cpu`가 나와도 오늘 실습은 전부 되고, 마지막 훈련만 조금 느리다.



PART 02

Mini-batch와 Shuffle로 데이터를 공급

2. Mini-batch: 나눠서 처리하는 이유



batch = 한 번에 처리하는 샘플 묶음, 배치마다 파라미터를 1회 갱신한다

- Mini-batch 학습이 표준인 이유 세 가지

- GPU 메모리(VRAM)에는 배치 하나만 올리면 되어 데이터셋이 아무리 커져도 처리할 수 있다.
- GPU는 배치로 묶어 줄 때 병렬 처리 성능이 나온다.
- 전체 계산을 기다리지 않고 배치마다 파라미터를 갱신하므로 같은 시간에 더 많이 학습한다.

2. 배치 수 계산

계산해 보기, 배치가 몇 개 생기나

```
total, batch = 60000, 64
print(total // batch)
print(total % batch)
print(937 + 1)
```

샘플 6만 개를 64개씩 배치로
→ 937 (가득 찬 배치)
→ 32 (마지막 배치의 샘플 수)
→ 938 (총 배치 수, 마지막은 덜 참)

- **batch size는 하이퍼파라미터**
 - 오늘은 **64**로 시작, 관례상 2의 거듭제곱(32, 64, 128, 256)을 많이 사용.
 - 마지막 배치는 개수가 모자라도 그대로 처리한다. (32개)
- **크기의 트레이드오프**
 - 크게 잡으면 epoch당 갱신 횟수가 줄어 같은 epoch에서 덜 학습된다.
 - 작게 잡으면 갱신은 잦지만 GPU 병렬 효율이 떨어진다.
 - 최적화를 찾는게 중요.

2. 핵심 용어 3개

용어	정의	오늘 수업에서
batch size	배치 하나에 담는 샘플 수	64
step (iteration)	배치 1개 처리 + 파라미터 1회 갱신	1 epoch에 938 step
epoch	전체 데이터를 1회 순회	3 epoch 학습 예정

- 지난주에 '걸음'이라 부른 것의 정식 명칭은 **step**
- 938 step을 마치면 60,000장을 전부 한 번 본 것이고 그것이 **1 epoch**다 ($937 \times 64 + 32$)

2. Shuffle: 데이터 순서가 학습을 바꾼다

정렬된 데이터, `shuffle=False` (클래스별 폴더로 수집된 상태)



배치가 한 클래스로만 채워져 기울기가 치우친다(좋지 않은 학습)

무작위 순서, `shuffle=True`



배치마다 클래스가 고르게 섞인다, 매 epoch 재배열

같은 데이터 6만 개, 공급 순서만 달라도 평가 정확도는 10% vs 93% (뒤에서 검증 예정)

- 정렬된 데이터로 학습하면

- 실무 데이터는 클래스별 폴더로 수집되는 일이 많아 그대로 읽으면 정렬 상태가 된다.
- 연속된 배치가 같은 클래스만 담으면 기울기가 한쪽으로 치우쳐 마지막 클래스만 남는다.
- `shuffle=True`는 매 epoch 시작마다 순서를 새로 섞는다. (10% vs 93%, 뒤에서 검증할 예정)

2. 질문, 학습 방식 진단

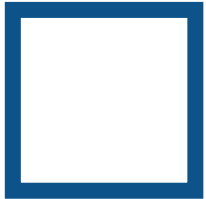
질문

1. 지난주처럼 60,000장을 full-batch로 학습하면 1 epoch 동안 파라미터는 몇 번 갱신될까?
2. 같은 데이터를 batch_size 64의 mini-batch로 바꾸면 1 epoch 동안 몇 번 갱신될까?
3. shuffle=True는 데이터를 언제 섞을까, 처음에 한 번? 아니면 매 epoch마다?

2. 예제 문제 풀이

문제	풀이	답
1	전체가 한 덩어리이므로 loss 계산도 갱신도 epoch당 한 번	1회
2	배치 938개가 지나가며 배치마다 갱신	938회 (같은 epoch에 938배)
3	한 번만 섞으면 순서를 외울 수 있어 의미가 줄어든다	매 epoch 시작마다 재배열

- 학습 로그에 loss가 자주 찍히는 이유도 이것, **step**마다 기록되기 때문



PART 03

Dataset으로 데이터와 전처리를 관리

3. Dataset의 인터페이스, 이미 사용

인터페이스	코드	반환
전체 샘플 수	<code>len(train_all)</code>	60000
i번째 샘플 조회 (인덱싱)	<code>x, y = train_all[i]</code>	(이미지, 라벨)이 쌍으로 반환

- 지난주 `mnist[0]`으로 손글씨 5를 꺼낸 것이 바로 **Dataset** 인덱싱이었다.
- 샘플과 라벨이 항상 쌍으로 반환된다, 입력과 정답을 함께 줘야 하는 지도학습의 구조.
- 내 이미지 폴더로 커스텀 **Dataset**을 작성하는 방법은 차후에 다룰 예정.

3. transform: 전처리를 Dataset에 등록

지난주에 손으로	오늘부터	수행 내용
<code>.float() / 255</code>	<code>transforms.ToTensor()</code>	0~255 정수 → 0~1 실수 스케일링
<code>permute</code> 로 채널 이동	(위와 동일, 한 번에)	PIL 이미지 → (채널, 세로, 가로) Tensor
학습 코드에서 매번 반복	Dataset 생성 시 한 번 등록	샘플을 꺼낼 때마다 자동 적용

TIP `Normalize`는 평균을 빼고 값의 폭을 맞춰 학습을 안정시키는 전처리이며, 사전학습 모델의 전용 전처리(`weights.transforms()`)도 같은 계열

3. 예제 2-1: Dataset 생성

예제 2-1. Dataset 생성과 확인, 네 줄이 검수표

```
import torch
from torchvision import datasets, transforms
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor()) # 전처리 등록

print(len(train_all))          # → 60000 (샘플 수)
x0, y0 = train_all[0]          # 0번 샘플과 라벨
print(type(x0).__name__, y0)   # → Tensor 5 (지난주엔 PIL 이미지였음)
print(x0.shape)                # → torch.Size([1, 28, 28]) (채널이 맨 앞)
print(x0.min().item(), x0.max().item()) # → 0.0 1.0 (스케일링 자동 적용)
```

• 확인 포인트

- `len`은 60000이 나오고, `train_all[0]`은 (이미지, 라벨) 쌍을 반환
- 지난주엔 PIL 이미지가 나왔는데 오늘은 **Tensor**가 바로 나옴 (transform 효과)
- `shape`이 (1, 28, 28), 흑백이라 채널 1이고 채널이 맨 앞
- 값이 이미 0~1로 스케일링되어 /255를 따로 하지 않아도 됨

3. 실습, 데이터셋 탐색

실습

1. 테스트 데이터셋(train=False)을 로드해 샘플 수 확인
2. 100번 샘플의 shape과 라벨 확인
3. 테스트 라벨 앞 1000개 중 7의 개수 세기

• 힌트

- 1. `datasets.MNIST(root="data", train=False, download=True, transform=...)` (train만 False로)
- 2. `x, y = test[100]` 그리고 `print(x.shape, y)`
- 3. `(test.targets[:1000] == 7).sum()`, 지난주 정확도 계산과 같은 비교 패턴

3. 실습, 정답 예시

실습 1. 정답 예시

```
import torch
from torchvision import datasets, transforms
test = datasets.MNIST(root="data", train=False, download=True,
                      transform=transforms.ToTensor()) # 1. 테스트 데이터셋
print(len(test)) # → 10000
x, y = test[100] # 2. 100번 샘플
print(x.shape, y) # → torch.Size([1, 28, 28]) 6
print((test.targets[:1000] == 7).sum().item()) # 3. 라벨 앞 1000개 중 7의 개수
# → 99
```

- 포인트

- `train=False` 하나로 테스트 데이터셋 (10,000개)이 로드.
- 3의 `(targets == 7).sum()`은 지난주 정확도 계산과 같은 비교 패턴.
- 라벨 분포를 세어 보는 것은 데이터 탐색(EDA)의 기본

- 다음

- 데이터셋 준비 끝
- `DataLoader`를 연결

3. 질문, 코드 결과 예측

질문

1. `train_all[59999]`를 실행하면 무엇이 반환될까? (샘플은 60,000개)
2. `transform` 없이 `datasets.MNIST(root="data", train=True)`로 로드하면, 0번 샘플의 이미지는 어떤 타입일까?
3. `x0.shape`이 `(1, 28, 28)`일 때 맨 앞의 1은 무엇을 의미할까?

3. 코드 결과 예측 답변

실행해서 검증

```
import torch
from torchvision import datasets, transforms
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())

x, y = train_all[59999]          # 1. 마지막 샘플 (인덱스는 0부터)
print(x.shape, y)               # → torch.Size([1, 28, 28]) 8
raw = datasets.MNIST(root="data", train=True, download=True) # 2. transform 없이 로드
print(type(raw[0][0]).__name__)  # → Image (PIL 이미지 원본)
```

• 풀이

- 인덱스는 0부터이므로 59999가 마지막 샘플, (이미지, 라벨 8) 쌍이 반환된다
- transform이 없으면 지난주처럼 PIL Image 원본이 반환된다
- 맨 앞 1은 채널 수다, 흑백이라 1이고 RGB 컬러 이미지면 3이 된다



PART 04

DataLoader로 배치를 만들어 공급

4. 예제 2-2: DataLoader 생성

예제 2-2. DataLoader 생성, 인자는 두 개

```
import torch
from torchvision import datasets, transforms
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())
from torch.utils.data import DataLoader
loader = DataLoader(train_all, batch_size=64, shuffle=True) # 로더 생성 한 줄
print(len(loader))      # → 938   (앞서 계산한 배치 수 그대로)
x, y = next(iter(loader)) # 첫 배치 하나 꺼내 보기 (관용구)
print(x.shape)           # → torch.Size([64, 1, 28, 28]) (첫 차원 = 배치 크기)
print(y.shape)           # → torch.Size([64])   (라벨도 64개 묶음)
print(y[:8].tolist())    # → ex. [3, 9, 0, 4, 1, 2, 5, 5] (섞여서 매번 다름)
```

- 출력 해석

- `len(loader)`가 938, 앞에서 계산한 배치 수 그대로다
- 배치 shape (64, 1, 28, 28)의 첫 차원이 배치 크기
- 라벨 `y`도 64개 묶음 Tensor로 함께 반환된다

- 관용구 하나

- `next(iter(loader))`는 첫 배치만 꺼내 보는 관용구, 학습 때는 `for x, y in loader:`로 순회한다

4. 배치 하나를 시각화

배치 시각화 (make_grid는 복사해서 쓰는 유틸리티)

```
import matplotlib.pyplot as plt
from torchvision.utils import make_grid
x, y = next(iter(loader))
grid = make_grid(x, nrow=8)
plt.figure(figsize=(6, 6))
plt.imshow(grid.permute(1, 2, 0))
plt.title(f"batch shape = {tuple(x.shape)}"); plt.axis("off"); plt.show()
print(y[:8].tolist())
```

배치를 격자 이미지로 (시각화 유틸)
배치 하나 (샘플 64개)
8개씩 8줄 격자
(채널, 세로, 가로) → 시각화 순서
첫 줄 8개의 라벨

batch shape = (64, 1, 28, 28)



4. 예제 2-3: shuffle 동작 검증

예제 2-3. 같은 데이터, 공급 순서만 다르게

```
import torch
from torchvision import datasets, transforms

train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())

from torch.utils.data import DataLoader
plain = DataLoader(train_all, batch_size=8, shuffle=False)
mixed = DataLoader(train_all, batch_size=8, shuffle=True)
print(next(iter(plain))[1].tolist())
# → [5, 0, 4, 1, 9, 2, 1, 3] (항상 저장되는 순서)
print(next(iter(mixed))[1].tolist())
# → ex. [7, 2, 8, 6, 0, 6, 4, 3] (섞임, 코드를 재실행 할 때마다 값 변경)
```

- 출력 해석
 - `shuffle=False`는 항상 [5, 0, 4, 1, ...], 저장 순서 그대로 반환
 - `shuffle=True`는 순회할 때마다 매번 다른 순서로 반환
- 기억할 것
 - 데이터 자체는 그대로, DataLoader가 공급 순서만 바꿈

4. 실습, batch size 바꿔 보기

실습

1. batch_size를 16, 256, 8192로 바꿔 가며 배치 수(len) 확인
2. batch_size=256의 첫 배치 shape 확인, 이어서

• 힌트

- ① `DataLoader(train_all, batch_size=크기, shuffle=True)` 만들고 `len(...)` 출력
- ② `x, y = next(iter(...))` 그리고 `print(x.shape)`

4. 실습, 정답 예시

실습 2. 정답 예시

```
import torch
from torchvision import datasets, transforms
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())
from torch.utils.data import DataLoader
for bs in (16, 256, 8192):
    ld = DataLoader(train_all, batch_size=bs, shuffle=True)
    print(bs, len(ld))
ld = DataLoader(train_all, batch_size=256, shuffle=True)
x, y = next(iter(ld))
print(x.shape)
```

- **포인트**

- 16이면 3750개, 256이면 235개, 8192면 8개 batch
- shape은 첫 차원만 바뀐다, 데이터 형식은 그대로고 배치 구성만 달라진 것

4. 질문, DataLoader 동작 예측

질문

1. DataLoader(train_all, batch_size=100, shuffle=False)의 len은 얼마일까?
2. 그 로더의 첫 배치 라벨 앞 4개는 무엇일까?
3. shuffle=True로 바꾸면 첫 배치의 라벨을 미리 맞힐 수 있을까?

4. DataLoader 동작 예측 답변

실행해서 검증

```
import torch
from torchvision import datasets, transforms
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())

from torch.utils.data import DataLoader
ld = DataLoader(train_all, batch_size=100, shuffle=False)
print(len(ld))                # 1. 60000 / 100 = 600배치
# → 600
print(next(iter(ld))[1][:4].tolist()) # 2. shuffle 없이는 저장 순서 그대로
# → [5, 0, 4, 1]
```

• 풀이

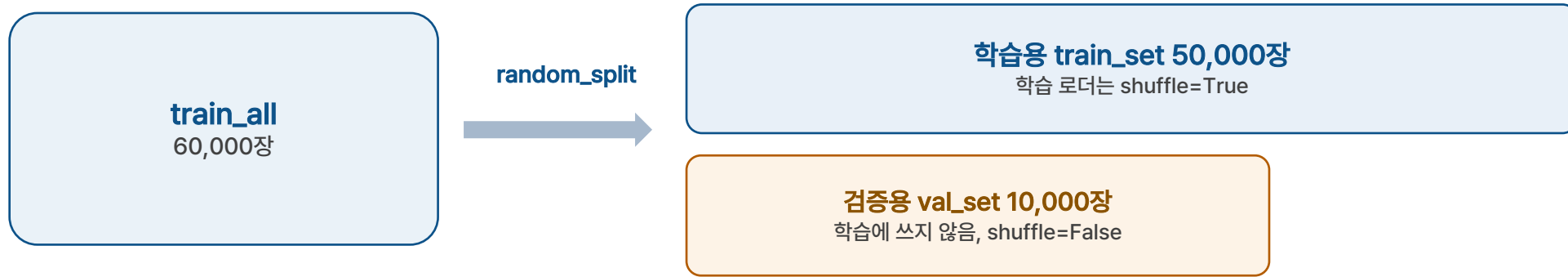
- $60,000 \div 100$ 이므로 600개 배치가 된다
- 섞지 않으면 저장 순서 그대로라 [5, 0, 4, 1], 이번 주에 계속 본 그 순서다
- 맞힐 수 없다, 순회마다 새로 섞이기 때문이며 그것이 shuffle의 목적이다



PART 05

학습과 검증 데이터를 분리 (Train, Val)

5. 분리하는 이유: 일반화 성능의 측정



학습에 쓰지 않은 데이터로 매 epoch 일반화 성능을 점검한다

- 학습에 쓴 데이터로 평가하면

- 100점이 나와도 일반화된 실력인지 단순 암기인지 구분할 수 없음
- 그래서 학습에 쓰지 않는 검증 데이터 10,000장을 떼어 매 epoch 끝에 Test
- `train=False` 테스트셋이 최종 평가용이라면 validation은 학습 중 점검용

5. 예제 2-4: random_split으로 분리

예제 2-4. 데이터 분리와 로더 두 개 생성

```
import torch
from torchvision import datasets, transforms
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())

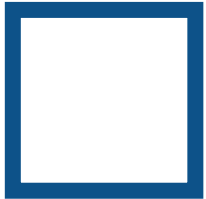
from torch.utils.data import DataLoader, random_split
g = torch.Generator().manual_seed(0)      # 분리 재현용 시드
train_set, val_set = random_split(train_all, [50000, 10000], generator=g)
print(len(train_set), len(val_set))        # → 50000 10000
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)    # 학습 로더, shuffle 켜
val_loader = DataLoader(val_set, batch_size=256, shuffle=False)      # 검증 로더, shuffle 끄
print(len(train_loader), len(val_loader))  # → 782 40
```

출력 해석

- `random_split`이 무작위로 50,000과 10,000으로 나눈다, 앞에서 자르면 수집 순서대로 치우칠 수 있기 때문
- `Generator().manual_seed(0)`은 분리 결과를 매번 똑같이 만드는 재현용 시드

로더 규칙

- 학습 로더는 `shuffle`을 켜고, 검증 로더는 평가만 하므로 `shuffle`을 사용하지 않음
- 배치 수가 학습 782, 검증 40으로 나오면 성공이다



PART 06

완성한 파이프라인으로 모델을 다시 학습

6. 지난주 방식 vs 오늘 방식

	지난 학습	이번 학습
데이터	6,000장만	50,000장 전부
공급 방식	full-batch 한 번에	batch 64 × 782개, shuffle
전처리	/255 직접	ToTensor() 자동
평가	학습 종료 후 한 번	매 epoch 검증 10,000장
모델과 학습 루프	2층 신경망, 4단계	그대로 (모델은 안 바꿈)

6. 예제 2-5: 학습 실행

예제 2-5. 파이프라인 학습, epoch 루프 안에 배치 루프

```
import torch.nn as nn
device = "cuda" if torch.cuda.is_available() else "cpu"
torch.manual_seed(0) # 재현용
model = nn.Sequential(nn.Linear(784, 100), nn.ReLU(),
                      nn.Linear(100, 10)).to(device) # 지난주 모델 그대로
opt = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.CrossEntropyLoss()
for epoch in range(3): # 3 epoch
    for x, y in train_loader: # 배치 782개 순회
        x, y = x.view(-1, 784).to(device), y.to(device) # 배치를 펴서 GPU로
        loss = loss_fn(model(x), y) # 지난주와 같은 학습 단계
        opt.zero_grad(); loss.backward(); opt.step()
    correct = 0 # epoch 끝마다 검증
    with torch.no_grad():
        for x, y in val_loader:
            x, y = x.view(-1, 784).to(device), y.to(device)
            correct += (model(x).argmax(dim=1) == y).sum().item()
    print(epoch + 1, round(correct / 10000 * 100, 1))
# → (예) 1 89.9
# → (예) 2 93.8
# → (예) 3 94.9
```

• 구조

- 바깥 for 루프가 3 epoch을 돌고, 안쪽 for루프가 배치 782개를 하나씩 받음
- 배치 하나를 처리하는 내용은 지난주 학습 루프 4단계 그대로다

• 결과

- 검증 정확도가 89.9 → 93.8 → 94.9로 올라 지난주 92%를 넘었다

6. 결과: 모델은 그대로, 데이터와 공급을 바꿨더니

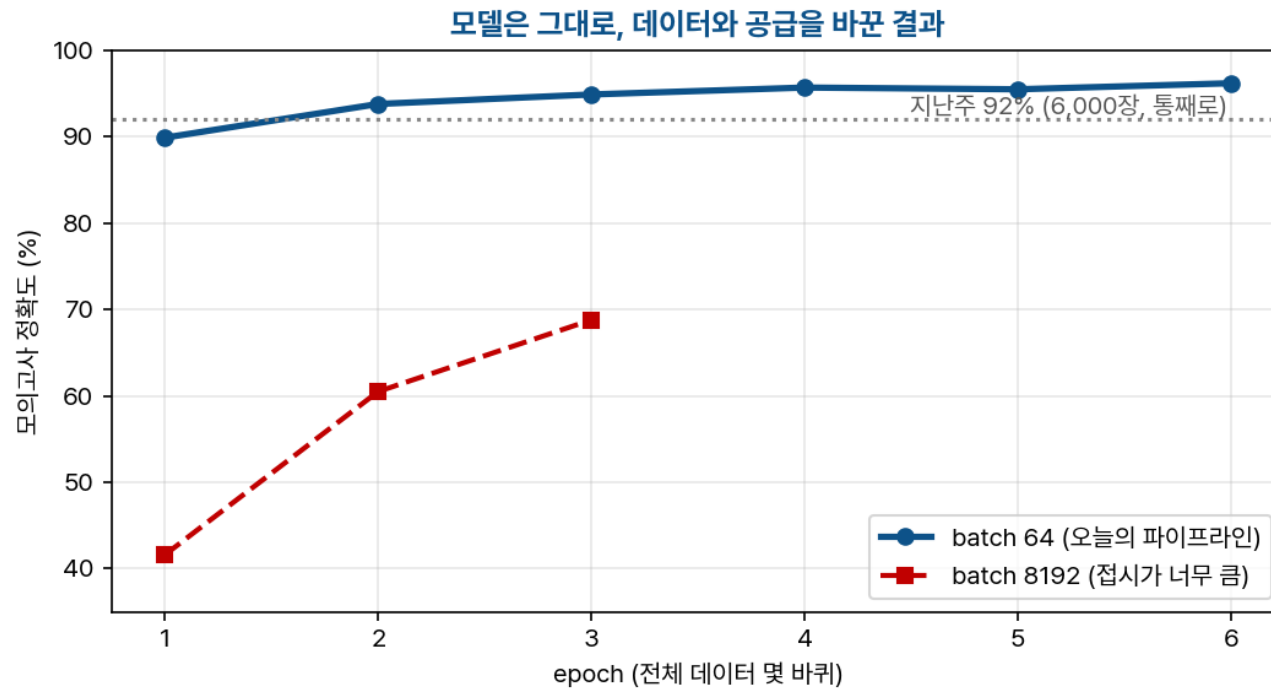


Fig. 2. epoch별 검증 정확도, 공급 방식이 성능을 바꾼다

• 그래프 해석

- 파란 선이 오늘의 파이프라인, 3 epoch에 94.9%를 찍고 더 학습하면 96%대까지 오름
- 빨간 선처럼 batch size를 8192로 키우면 갱신 횟수가 부족해 한참 아래에 머뭄

• 결론

- 모델은 그대로인데 성능이 올랐다, 데이터의 양과 공급 방식이 성능을 결정한 것

6. 실습, 공급 설정 실험

실습

1. 예제 2-4의 train_loader에서 batch_size를 8192로 변경후 결과 확인
2. 원래대로 코드로 돌리고 예제 2-5를 range(6)으로 6 epoch변경 후 결과 확인
3. 이어서, train_loader의 shuffle을 False로 한 후 결과 확인

실험	바꾼 설정
기준	batch 64, 3 epoch, shuffle=True
1	batch_size=8192
2	6 epoch
3	shuffle=False

6. 실습, 정답 예시

- 관찰 결과 세 가지
 - batch size가 너무 크면 갱신 횟수가 부족해 같은 epoch에서 덜 학습된다
 - epoch을 늘리면 더 학습되어 96%대에 이른다
 - 학습 로더의 `shuffle=True`는 필수

6. 질문, 파이프라인 코드 진단

질문

1. random_split 대신 train_all을 앞에서부터 50,000개 잘라 학습용으로 쓰면 어떤 위험이 있을까?
2. 검증 루프에서 with torch.no_grad(): 를 빼먹으면 정확도가 틀려질까, 아니면 다른 문제가 생길까?
3. 검증 로더에 shuffle=True를 쓰면 검증 정확도가 달라질까?

6. 파이프라인 코드 진단 답변

문제	풀이	답
1	수집 순서나 라벨 순으로 저장된 데이터면 학습과 검증의 분포가 달라진다	무작위 분리가 안전
2	정확도는 같지만 기울기 추적이 켜져 메모리와 시간을 낭비한다	평가에는 <code>no_grad()</code> 가 표준
3	평가 지표는 데이터 순서와 무관하다	달라지지 않는다, 관례상 False

- 학습 로더의 shuffle은 성능을 바꾸고, 검증 로더의 shuffle은 아무것도 바꾸지 않는다

6. 정리

오늘의 구성 요소	역할	핵심 한 줄
Dataset + transform	데이터 보관과 자동 전처리	len과 [i], 조회할 때마다 전처리 적용
DataLoader	shuffle과 배치 구성	batch_size=64, shuffle=True
batch / step / epoch	묶음 / 갱신 1회 / 전체 1회 순회	782 step = 50,000장 1 epoch
random_split	학습과 검증 데이터 분리	50,000 + 10,000, 검증 로더는 shuffle 끄

- 다음 시간, 신경망 직접 구현

- 지금까지 완성품으로 쓴 `nn.Sequential` 내부를 열어, 선형 계층과 활성화 함수를 직접 쌓아볼 예정